# Outline

Introduction

String Diagrams

Kullback–Leibler Divergence

Quantitative Diagrammatic Axiomatisation

# Algebraic Semantics, Briefly

*Idea:* programming syntax as algebraic operations (pioneered in [SS71] and [Gog+77]).

$$;(P, Q) = P \,;\, Q \qquad\qquad \mathsf{ifte}(P, Q, Q') = \mathsf{if}\ P\ \mathsf{then}\ Q\ \mathsf{else}\ Q'$$

This enables equational reasoning for program equivalence:

$$\dfrac{\dfrac{P \,;\, (Q \,;\, R) = (P \,;\, Q) \,;\, R \qquad\qquad \mathsf{ifte}(P, Q, Q) = P \,;\, Q}{\vdots}}{\mathsf{ifte}(P, Q, Q) \,;\, R = \mathsf{ifte}(P, Q \,;\, R, Q \,;\, R)}$$

...among other things.

[SS71] Dana Scott and Christopher Strachey. "Toward a Mathematical Semantics for Computer Languages". In: *Proceedings of the Symposium on Computers and Automata.* Vol. 21. 1971

[Gog+77] J. A. Goguen et al. "Initial Algebra Semantics and Continuous Algebras". In: *J. ACM* 24.1 (1977), 68–95. ISSN: 0004-5411. DOI: 10.1145/321992.321997. URL: https://doi.org/10.1145/321992.321997

# Problems with Randomness 1: Approximations

Example ([Neu51])

```
return flip(0.5)
```

```
do
x = flip(p)
y = flip(p)
while (x == y)
return x
```

As long as the bias is consistent and not total ($0\% < p < 100\%$), the two programs have the same behavior.

[Neu51] John von Neumann. "Various Techniques Used in Connection with Random Digits". In: *National Bureau of Standards Applied Math Series*. Ed. by G. E. Forsythe. Vol. 12. Reprinted in von Neumann's Collected Works, Volume 5, Pergamon Press, 1963, pp. 768–770. National Bureau of Standards, 1951, pp. 36–38

# Problems with Randomness 1: Approximations

## Example (Guaranteed Termination)

```
return flip(0.5)
```

```
i = 0
do
i = i + 1
x = flip(p)
y = flip(p)
while (x == y) AND i <= 1000
return x
```

The second program is very close to being a fair coin flip. We need a finer comparison than exact equivalence.

# Problems with Randomness 2: Substitution

**Example (Reusing results)**

```
x = flip(0.5)
return x or x
```

```
return flip(0.5) or flip(0.5)
```

The semantics of these two programs are two *different* distributions:

$$\begin{bmatrix}0.5\\0.5\end{bmatrix} \qquad \begin{bmatrix}0.75\\0.25\end{bmatrix}$$

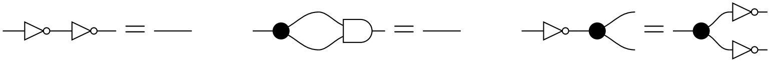
We need a *resource sensitive* language to reason about these programs.

# Overview

Our solution is quantitative diagrammatic reasoning. It combines quantitative equations and string diagrams.

## String Diagrams.

- Processes are morphisms in monoidal categories
- String diagrams represent such morphisms
- Reason equationally:



## Quantitative Equations.

- Replace exact equivalence with a distance
- Reason equationally: $P =_\varepsilon Q$ means $P$ and $Q$ are at distance $\leq \varepsilon$.

# Quantitative Diagrammatic Axiomatisation of Divergence

Ralph Sarkis (he/him)

joint work with Fabio Zanasi

July 6th, 2026

# Outline

Introduction

String Diagrams

Kullback–Leibler Divergence

Quantitative Diagrammatic Axiomatisation

# Intuition

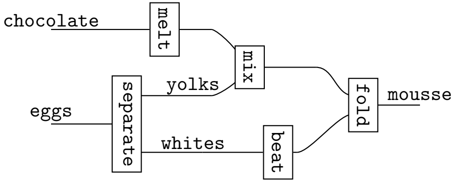
String diagrams are a syntax for processes and systems.

## Example

# Intuition

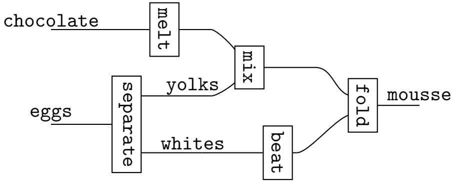
String diagrams are a syntax for processes and systems.

## Example



String diagrams are a syntax for processes and systems.

## Example

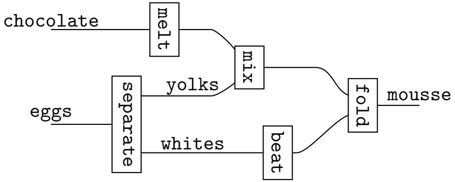



`fold` ( , )

# Intuition

String diagrams are a syntax for processes and systems.

## Example

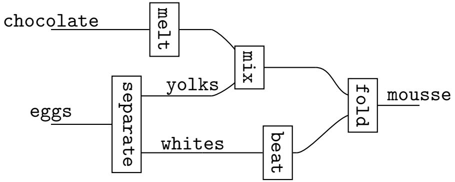



$\texttt{fold}\,(\texttt{mix}( \qquad\qquad\qquad , \qquad\qquad\qquad ), \qquad\qquad\qquad )$

# Intuition

String diagrams are a syntax for processes and systems.

## Example

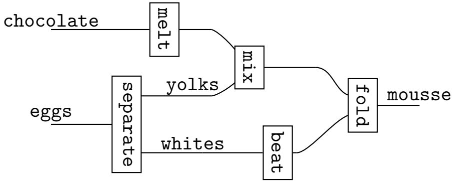



`fold (mix(melt(chocolate),                    ),                    )`

# Intuition

String diagrams are a syntax for processes and systems.

## Example

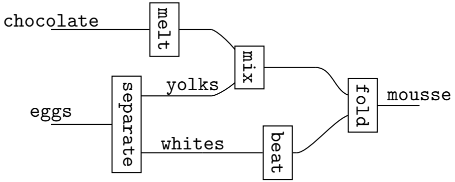



$\texttt{fold}\,(\texttt{mix}(\texttt{melt}(\texttt{chocolate}), \quad (\texttt{separate}(\texttt{eggs}))), \qquad\qquad\qquad\qquad )$

# Intuition

String diagrams are a syntax for processes and systems.

## Example

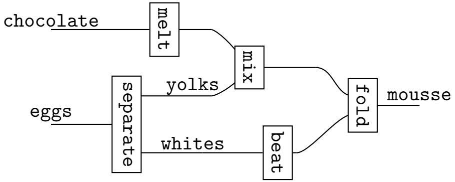



$$\texttt{fold}\,(\texttt{mix}(\texttt{melt}(\texttt{chocolate}), \pi_1(\texttt{separate}(\texttt{eggs}))), \qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad )$$

# Intuition

String diagrams are a syntax for processes and systems.

## Example

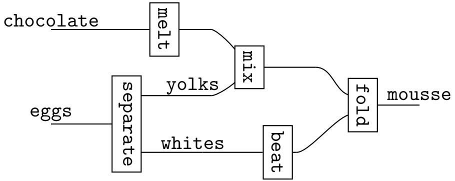



$$\mathtt{fold}\,(\mathtt{mix}(\mathtt{melt}(\mathtt{chocolate}), \pi_1(\mathtt{separate}(\mathtt{eggs}))), \mathtt{beat}(\pi_2(\mathtt{separate}(\mathtt{eggs}))))$$

# Intuition

String diagrams are a syntax for processes and systems.

## Example



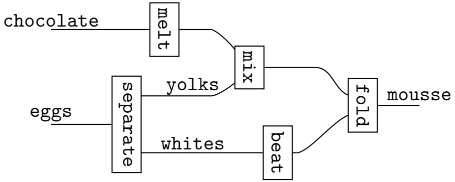

$$\mathtt{fold}\,(\mathtt{mix}(\mathtt{melt}(\mathtt{chocolate}), \pi_1(\mathtt{separate}(\mathtt{eggs}))), \mathtt{beat}(\pi_2(\mathtt{separate}(\mathtt{eggs}))))$$

$$(\mathtt{melt} \otimes \mathtt{separate}); (\mathtt{mix} \otimes \mathtt{beat}); \mathtt{fold}$$

# Intuition

String diagrams are a syntax for processes and systems.

## Example




$$\texttt{fold}\,(\texttt{mix}(\texttt{melt}(\texttt{chocolate}), \pi_1(\texttt{separate}(\texttt{eggs}))), \texttt{beat}(\pi_2(\texttt{separate}(\texttt{eggs}))))$$

$$(\texttt{melt} \otimes \texttt{separate}); (\texttt{mix} \otimes \texttt{beat}); \texttt{fold}$$

$$(\mathrm{id} \otimes \texttt{separate}); (((\texttt{melt} \otimes \mathrm{id}); \texttt{mix}) \otimes \texttt{beat}); \texttt{fold}$$

# Monoidal Terms

A **generator** is a symbol with an arity $u$ and a coarity $v$ in $\mathbb{N}$: $u \;-\!\boxed{f}\!-\; v$.
With a set of generators $\Sigma$, inductively generate the set of **monoidal terms** $\mathcal{M}_\Sigma$.

$$\frac{u \;-\!\boxed{f}\!-\; v \in \Sigma}{u \;-\!\boxed{f}\!-\; v \in \mathcal{M}_\Sigma}\ \textsc{Gen} \qquad \frac{x, y \in \mathbb{N}}{\begin{smallmatrix} x \\ y \end{smallmatrix} \times \begin{smallmatrix} y \\ x \end{smallmatrix} \in \mathcal{M}_\Sigma}\ \textsc{Swap} \qquad \frac{x \in \mathbb{N}}{x \;-\!\!-\; x \in \mathcal{M}_\Sigma}\ \textsc{Id} \qquad \frac{}{\cdots\cdots \in \mathcal{M}_\Sigma}\ \textsc{Unit}$$

$$\frac{u \;-\!\boxed{f}\!-\; v \qquad v \;-\!\boxed{g}\!-\; w}{u \;-\!\boxed{f}\!-\!\boxed{g}\!-\; w}\ \textsc{Seq} \qquad \frac{u \;-\!\boxed{f}\!-\; v \qquad u' \;-\!\boxed{f'}\!-\; v'}{\begin{matrix} u \;-\!\boxed{f}\!-\; v \\ u' \;-\!\boxed{f'}\!-\; v' \end{matrix}}\ \textsc{Par}$$

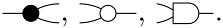
# Monoidal Terms

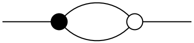
A **generator** is a symbol with an arity $u$ and a coarity $v$ in $\mathbb{N}$: $u \,-\!\boxed{f}\!-\, v$.
With a set of generators $\Sigma$, inductively generate the set of **monoidal terms** $\mathcal{M}_\Sigma$.

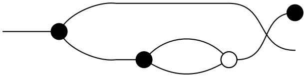
$$\frac{u \,-\!\boxed{f}\!-\, v \in \Sigma}{u \,-\!\boxed{f}\!-\, v \in \mathcal{M}_\Sigma}\ \textsc{Gen} \qquad \frac{x, y \in \mathbb{N}}{\begin{smallmatrix}x\\y\end{smallmatrix} \times \begin{smallmatrix}y\\x\end{smallmatrix} \in \mathcal{M}_\Sigma}\ \textsc{Swap} \qquad \frac{x \in \mathbb{N}}{x \,-\!-\, x \in \mathcal{M}_\Sigma}\ \textsc{Id} \qquad \frac{}{\cdots\cdots \in \mathcal{M}_\Sigma}\ \textsc{Unit}$$

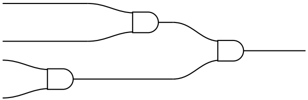
$$\frac{u \,-\!\boxed{f}\!-\, v \qquad v \,-\!\boxed{g}\!-\, w}{u \,-\!\boxed{f}\!-\!\boxed{g}\!-\, w}\ \textsc{Seq} \qquad \frac{u \,-\!\boxed{f}\!-\, v \qquad u' \,-\!\boxed{f'}\!-\, v'}{\begin{matrix}u \,-\!\boxed{f}\!-\, v\\ u' \,-\!\boxed{f'}\!-\, v'\end{matrix}}\ \textsc{Par}$$

## Examples

With generators 





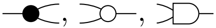


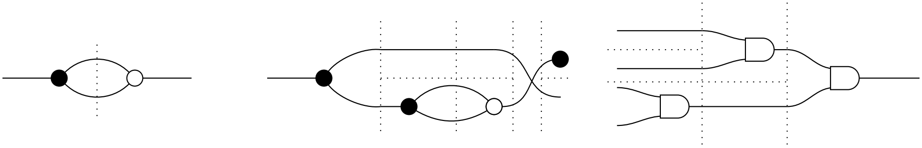
A **generator** is a symbol with an arity $u$ and a coarity $v$ in $\mathbb{N}$: $u \,-\!\boxed{f}\!-\, v$.
With a set of generators $\Sigma$, inductively generate the set of **monoidal terms** $\mathcal{M}_\Sigma$.

$$\frac{u \,-\!\boxed{f}\!-\, v \in \Sigma}{u \,-\!\boxed{f}\!-\, v \in \mathcal{M}_\Sigma}\ \textsc{Gen} \qquad \frac{x, y \in \mathbb{N}}{\begin{smallmatrix}x\\ y\end{smallmatrix} \times \begin{smallmatrix}y\\ x\end{smallmatrix} \in \mathcal{M}_\Sigma}\ \textsc{Swap} \qquad \frac{x \in \mathbb{N}}{x \;—\; x \in \mathcal{M}_\Sigma}\ \textsc{ID} \qquad \frac{}{\cdots\cdots \in \mathcal{M}_\Sigma}\ \textsc{Unit}$$

$$\frac{u \,-\!\boxed{f}\!-\, v \qquad v \,-\!\boxed{g}\!-\, w}{u \,-\!\boxed{f}\!-\!\boxed{g}\!-\, w}\ \textsc{Seq} \qquad \frac{u \,-\!\boxed{f}\!-\, v \qquad u' \,-\!\boxed{f'}\!-\, v'}{\begin{matrix}u \,-\!\boxed{f}\!-\, v\\ u' \,-\!\boxed{f'}\!-\, v'\end{matrix}}\ \textsc{Par}$$

## Examples

With generators .



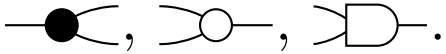
# Monoidal Terms

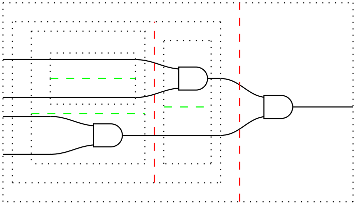
A **generator** is a symbol with an arity $u$ and a coarity $v$ in $\mathbb{N}$: $u \;—\boxed{f}—\; v$.
With a set of generators $\Sigma$, inductively generate the set of **monoidal terms** $\mathcal{M}_\Sigma$.

$$\frac{u \;—\boxed{f}—\; v \in \Sigma}{u \;—\boxed{f}—\; v \in \mathcal{M}_\Sigma}\ \textsc{Gen} \qquad \frac{x, y \in \mathbb{N}}{\begin{smallmatrix} x \\ y \end{smallmatrix} \times \begin{smallmatrix} y \\ x \end{smallmatrix} \in \mathcal{M}_\Sigma}\ \textsc{Swap} \qquad \frac{x \in \mathbb{N}}{x \;—\; x \in \mathcal{M}_\Sigma}\ \textsc{Id} \qquad \frac{}{\cdots\cdots \in \mathcal{M}_\Sigma}\ \textsc{Unit}$$

$$\frac{u \;—\boxed{f}—\; v \qquad v \;—\boxed{g}—\; w}{u \;—\boxed{f}—\boxed{g}—\; w}\ \textsc{Seq} \qquad \frac{u \;—\boxed{f}—\; v \qquad u' \;—\boxed{f'}—\; v'}{\begin{matrix} u \;—\boxed{f}—\; v \\ u' \;—\boxed{f'}—\; v' \end{matrix}}\ \textsc{Par}$$

## Examples

With generators .



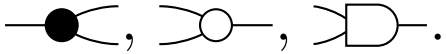
# Monoidal Terms

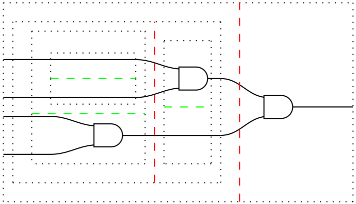
A **generator** is a symbol with an arity $u$ and a coarity $v$ in $\mathbb{N}$: $u \;-\!\boxed{f}\!-\; v$.
With a set of generators $\Sigma$, inductively generate the set of **monoidal terms** $\mathcal{M}_\Sigma$.

$$\frac{u \;-\!\boxed{f}\!-\; v \in \Sigma}{u \;-\!\boxed{f}\!-\; v \in \mathcal{M}_\Sigma}\ \textsc{Gen} \qquad \frac{x, y \in \mathbb{N}}{\begin{smallmatrix} x \\ y \end{smallmatrix} \times \begin{smallmatrix} y \\ x \end{smallmatrix} \in \mathcal{M}_\Sigma}\ \textsc{Swap} \qquad \frac{x \in \mathbb{N}}{x \;—\; x \in \mathcal{M}_\Sigma}\ \textsc{Id} \qquad \frac{}{\cdots\cdots \in \mathcal{M}_\Sigma}\ \textsc{Unit}$$

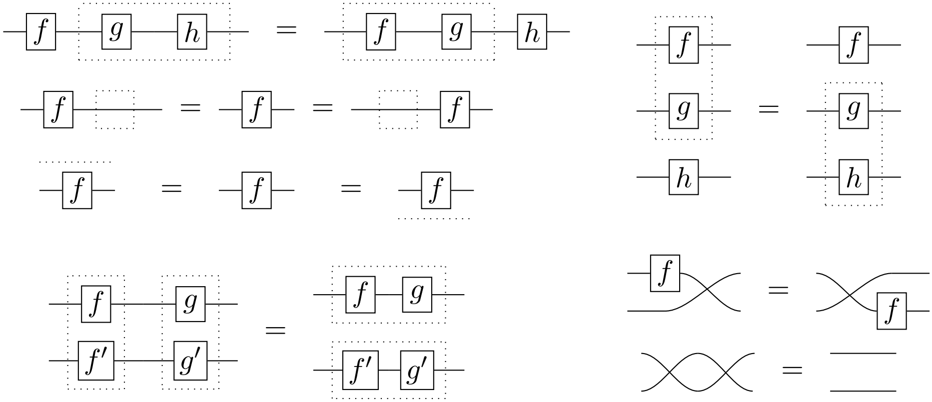
$$\frac{u \;-\!\boxed{f}\!-\; v \qquad v \;-\!\boxed{g}\!-\; w}{u \;-\!\boxed{f}\!-\!\boxed{g}\!-\; w}\ \textsc{Seq} \qquad \frac{u \;-\!\boxed{f}\!-\; v \qquad u' \;-\!\boxed{f'}\!-\; v'}{\begin{matrix} u \;-\!\boxed{f}\!-\; v \\ u' \;-\!\boxed{f'}\!-\; v' \end{matrix}}\ \textsc{Par}$$

## Examples

With generators .

 $= (((\mathrm{id} \otimes \mathrm{id}) \otimes \mathsf{and}) \,;\, (\mathsf{and} \otimes \mathrm{id})) \,;\, \mathsf{and}$

**String diagrams** are monoidal terms considered modulo the following equations.

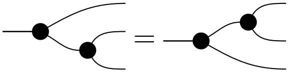


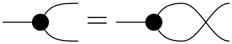

By [JS91], they are considered modulo continuous deformations.

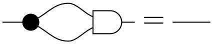
[JS91] André Joyal and Ross Street. "The geometry of tensor calculus. I". In: *Adv. Math.* 88.1 (1991), pp. 55–112. ISSN: 0001-8708,1090-2082. DOI: 10.1016/0001-8708(91)90003-P. URL: https://doi.org/10.1016/0001-8708(91)90003-P

# Equations

We can impose further equations between terms of the same type, e.g.,

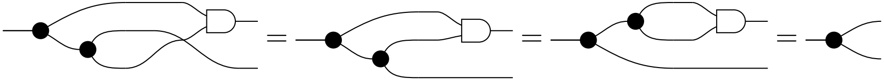






Diagrammatic reasoning consists of:

- Treating string diagrams up to **visually sound manipulations**.
- Applying **equational reasoning** by replacing diagrams with other diagrams that are known to be equal.



# Presentations

**Definition**

Given a set of generators $\Sigma$ and a set of equations $E$: define for any $u, v \in \mathbb{N}$,

$$\mathcal{S}_{\Sigma,E}(u,v) = \left\{ u \; - \boxed{f} - \; v \text{ built with } \Sigma \right\} / \text{diagrammatic reasoning with } E.$$

This freely generates the **syntactic category** $\mathcal{S}_{\Sigma,E}$.

### Definition

Given a set of generators $\Sigma$ and a set of equations $E$: define for any $u, v \in \mathbb{N}$,

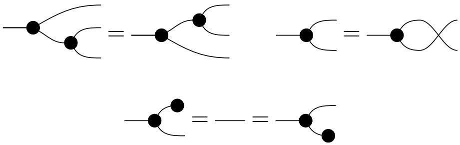
$$\mathcal{S}_{\Sigma,E}(u,v) = \left\{ u \ -\!\boxed{f}\!-\ v \text{ built with } \Sigma \right\} / \text{diagrammatic reasoning with } E.$$

This freely generates the **syntactic category** $\mathcal{S}_{\Sigma,E}$.
A **presentation** for a category $\mathbf{C}$ is a pair $(\Sigma, E)$ such that $\mathcal{S}_{\Sigma,E} \cong \mathbf{C}$.

$$\text{Morally,} \qquad \frac{\text{string diagrams } u \ -\!\boxed{f}\!-\ v}{\overline{\text{morphisms } f : u \to v \in \mathbf{C}}}$$

# Example 1: Finite Sets and Functions

*Generators*



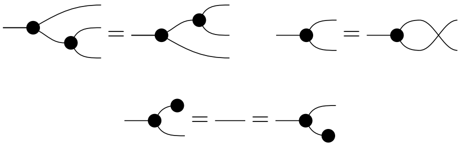
*Equations*

$\llbracket$  $\rrbracket : \{0, 1\} \to \{0\} = 0 \mapsto 0, 1 \mapsto 0$ $\llbracket$  $\rrbracket : \{\} \to \{0\} = \_ \mapsto$

*Semantics*

# Example 1: Finite Sets and Functions

*Generators*

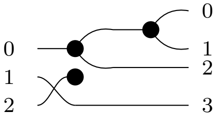


*Equations*

$[\![$  $]\!] : \{0,1\} \to \{0\} = 0 \mapsto 0, 1 \mapsto 0$ $[\![$  $]\!] : \{\} \to \{0\} = \_ \mapsto$

*Semantics*

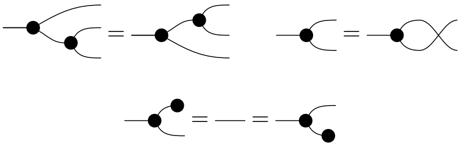
## Example



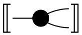
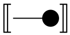

$$\iff \begin{cases} 0,1,2 & \mapsto 0 \\ 3 & \mapsto 1 \end{cases}$$

# Example 1: Finite Sets and Functions

*Generators*

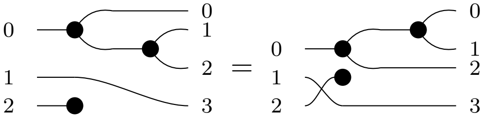


*Equations*

 $: \{0,1\} \to \{0\} = 0 \mapsto 0, 1 \mapsto 0$  $: \{\} \to \{0\} = \_ \mapsto$

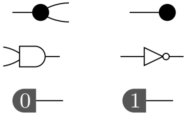
*Semantics*

## Example

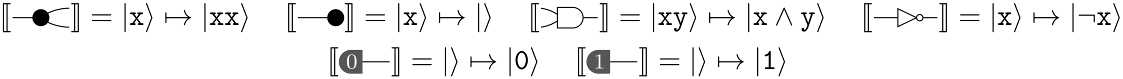



$$\iff \begin{cases} 0,1,2 & \mapsto 0 \\ 3 & \mapsto 1 \end{cases}$$

# Example 2: Boolean Circuits

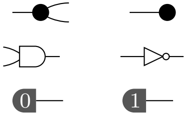



*Generators*

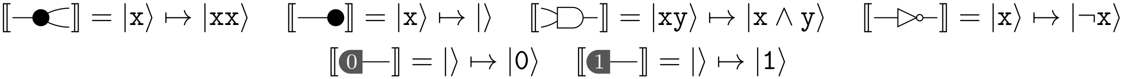



*Semantics*

# Example 2: Boolean Circuits



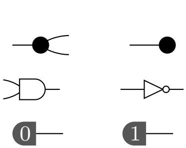
*Generators*



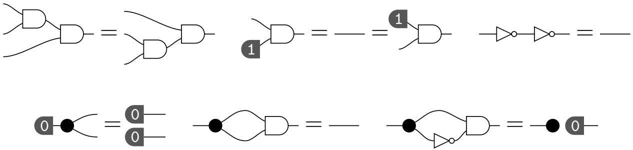

*Semantics*

$$\frac{u \;-\!\!\boxed{f}\!\!-\; v}{f : \mathbb{B}^u \to \mathbb{B}^v}$$

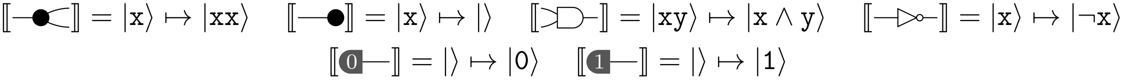
# Example 2: Boolean Circuits



*Generators*



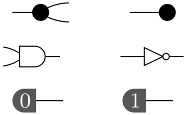

*Some equations*



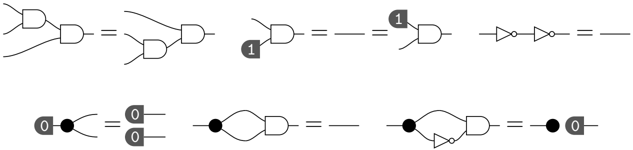

*Semantics*

$$\frac{u \;\text{—}\boxed{f}\text{—}\; v}{f : \mathbb{B}^u \to \mathbb{B}^v}$$

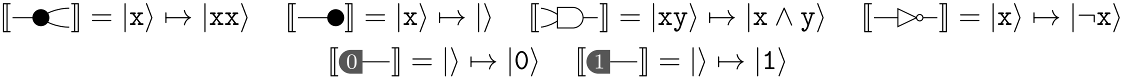
# Example 2: Boolean Circuits




*Generators*

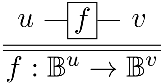


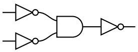

*Some equations*

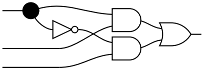



*Semantics*

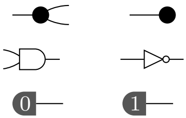
## Examples



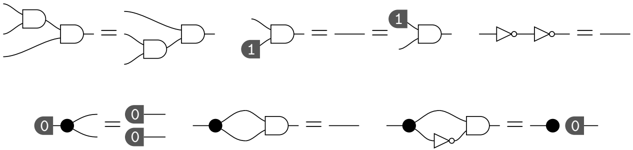





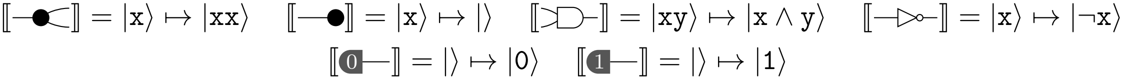
# Example 2: Boolean Circuits




*Generators*



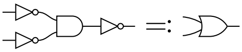

*Some equations*

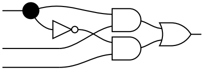



*Semantics*

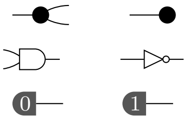
## Examples

$$\frac{u \;-\!\boxed{f}\!-\; v}{f : \mathbb{B}^u \to \mathbb{B}^v}$$

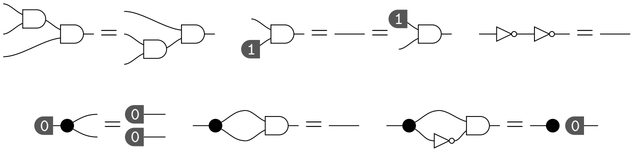


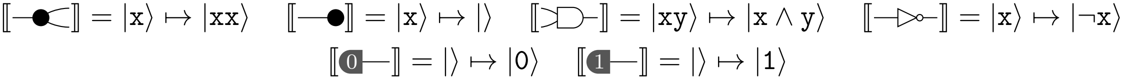
# Example 2: Boolean Circuits




*Generators*



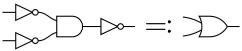

*Some equations*

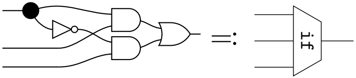



*Semantics*

## Examples

$$\frac{u \;—\boxed{f}—\; v}{f : \mathbb{B}^u \to \mathbb{B}^v}$$



$$\mathsf{BoolCirc} +$$

$$\boxed{p}\!-\!\!-\ ,\ p \in (0,1)$$

*Generators*

$$[\![\boxed{p}\!-\!\!-]\!] = |\rangle \mapsto p|\mathsf{1}\rangle + (1-p)|\mathsf{0}\rangle$$

*Semantics*

$$\dfrac{\dfrac{\text{string diagram } u \ -\!\boxed{f}\!-\ v}{\text{probabilistic map } f : \mathbb{B}^u \to \mathbb{B}^v}}{\text{function } f : \mathbb{B}^u \to \mathcal{D}(\mathbb{B}^v)}$$

# Example 3: Probabilistic Boolean Circuits

BoolCirc +

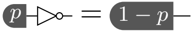
$p$ — , $p \in (0,1)$

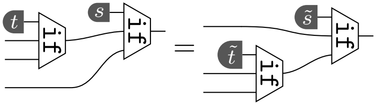
*Generators*

$$[\![ \; p \; ]\!] = |\rangle \mapsto p|1\rangle + (1-p)|0\rangle$$

*Semantics*

BoolCirc +






*Some equations*

$$\frac{\dfrac{\text{string diagram } u \;\boxed{f}\; v}{\text{probabilistic map } f : \mathbb{B}^u \to \mathbb{B}^v}}{\text{function } f : \mathbb{B}^u \to \mathcal{D}(\mathbb{B}^v)}$$

# Example 3: Probabilistic Boolean Circuits

$\mathsf{BoolCirc}$ +

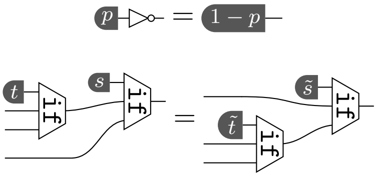
$\boxed{p}$— , $p \in (0, 1)$

*Generators*

$[\![\, \boxed{p}\!-\!\!- \,]\!] = |\rangle \mapsto p|\mathsf{1}\rangle + (1-p)|\mathsf{0}\rangle$

*Semantics*

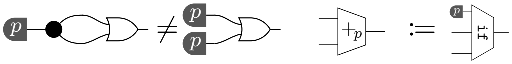
$\mathsf{BoolCirc}$ +




*Some equations*

## Examples

$$\frac{\text{string diagram } u \;-\!\boxed{f}\!-\; v}{\dfrac{\text{probabilistic map } f : \mathbb{B}^u \to \mathbb{B}^v}{\text{function } f : \mathbb{B}^u \to \mathcal{D}(\mathbb{B}^v)}}$$



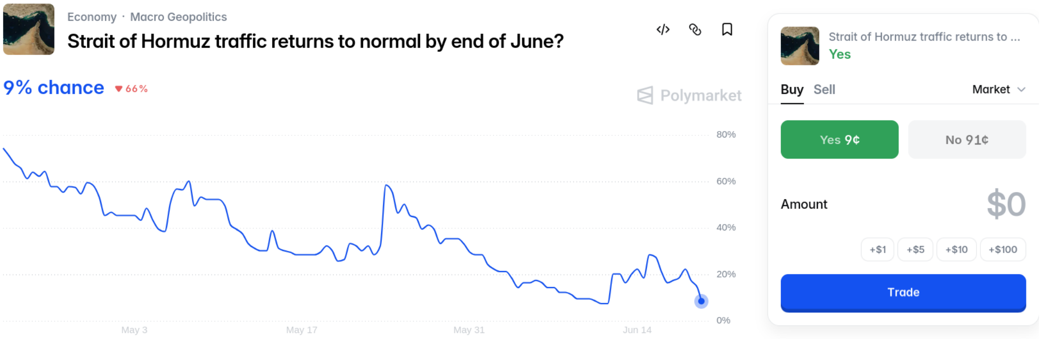
# Outline

Introduction

String Diagrams

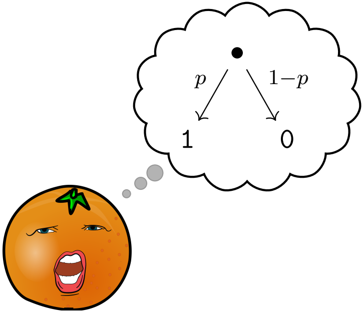
Kullback–Leibler Divergence

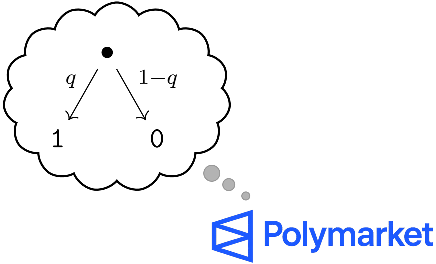
Quantitative Diagrammatic Axiomatisation




Buy Yes for $q$\$ or No for $(1-q)$\$. Reward is 1\$

# Betting with Insider Knowledge

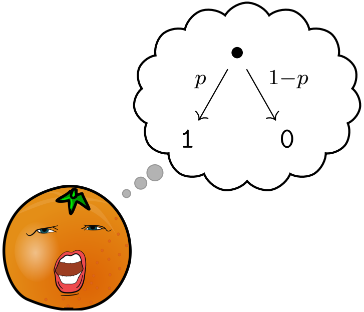


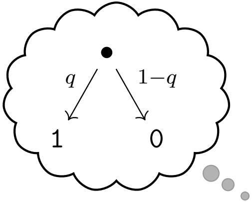




- Bad measure

$$\mathbb{E}[\mathsf{gain\_mult}] = p\frac{\mathsf{bet}(1)}{q} + (1-p)\frac{\mathsf{bet}(0)}{1-q},\ \ \mathsf{bet}(\mathtt{x}) \text{ is the proportion of money bet on } \mathtt{x}$$

# Betting with Insider Knowledge




Polymarket

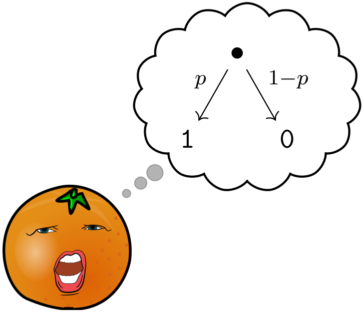
- Bad measure

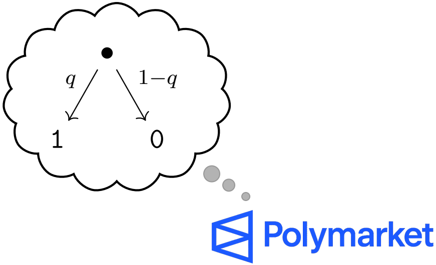
$$\mathbb{E}[\mathsf{gain\_mult}] = p\frac{\mathsf{bet}(1)}{q} + (1-p)\frac{\mathsf{bet}(0)}{1-q}, \ \mathsf{bet}(\mathtt{x}) \text{ is the proportion of money bet on } \mathtt{x}$$

- Better measure

$$\mathbb{E}[\log(\mathsf{gain\_mult})] = p\log\frac{\mathsf{bet}(1)}{q} + (1-p)\log\frac{\mathsf{bet}(0)}{1-q}$$

# Betting with Insider Knowledge




- Bad measure

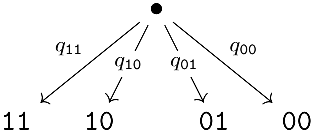
$$\mathbb{E}[\mathsf{gain\_mult}] = p\frac{\mathsf{bet}(1)}{q} + (1-p)\frac{\mathsf{bet}(0)}{1-q},\ \ \mathsf{bet}(\mathtt{x}) \text{ is the proportion of money bet on } \mathtt{x}$$

- Better measure

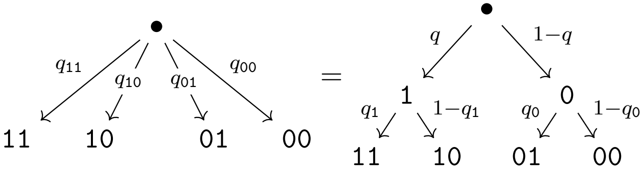
$$\max \mathbb{E}[\log(\mathsf{gain\_mult})] = p \log \frac{p}{q} + (1-p)\log\frac{1-p}{1-q} = \mathsf{kl}\left(\begin{bmatrix} p \\ 1-p \end{bmatrix}, \begin{bmatrix} q \\ 1-q \end{bmatrix}\right)$$

This is the **Kullback–Leibler divergence** between probability distributions.

# Bets with More Outcomes

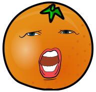


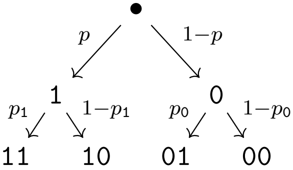
# Bets with More Outcomes



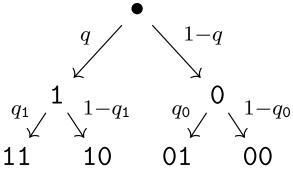

This is the **chain rule** of probability distributions.

# Bets with More Outcomes



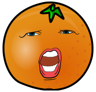


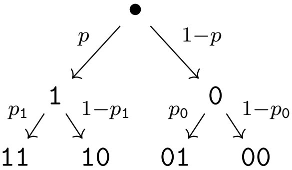

Polymarket



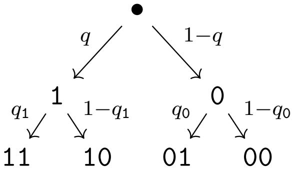

$$\max \mathbb{E}[\log(\mathsf{gain\_mult})] = \mathsf{kl}\left(\begin{bmatrix} p \\ 1-p \end{bmatrix}, \begin{bmatrix} q \\ 1-q \end{bmatrix}\right) + p\,\mathsf{kl}\left(\begin{bmatrix} p_1 \\ 1-p_1 \end{bmatrix}, \begin{bmatrix} q_1 \\ 1-q_1 \end{bmatrix}\right) + (1-p)\,\mathsf{kl}\left(\begin{bmatrix} p_0 \\ 1-p_0 \end{bmatrix}, \begin{bmatrix} q_0 \\ 1-q_0 \end{bmatrix}\right)$$

# Bets with More Outcomes






Polymarket




$$\mathsf{kl}\left(\begin{bmatrix} p_{11} \\ p_{10} \\ p_{01} \\ p_{00} \end{bmatrix}, \begin{bmatrix} p_{11} \\ p_{10} \\ p_{01} \\ p_{00} \end{bmatrix}\right) = \mathsf{kl}\left(\begin{bmatrix} p \\ 1-p \end{bmatrix}, \begin{bmatrix} q \\ 1-q \end{bmatrix}\right) + p\,\mathsf{kl}\left(\begin{bmatrix} p_1 \\ 1-p_1 \end{bmatrix}, \begin{bmatrix} q_1 \\ 1-q_1 \end{bmatrix}\right) + (1-p)\,\mathsf{kl}\left(\begin{bmatrix} p_0 \\ 1-p_0 \end{bmatrix}, \begin{bmatrix} q_0 \\ 1-q_0 \end{bmatrix}\right)$$

This is the **chain rule** of KL divergence.

# Recursive Definition of KL Divergence

Given

- Let $\mathbb{B} := \{\mathtt{0}, \mathtt{1}\}$ and $\varphi, \psi$ be distributions on $\mathbb{B}^n$,
- $p = \sum_y \varphi(\mathtt{1}y)$ and $q = \sum_y \psi(\mathtt{1}y)$ are the probabilities that the first bit is $\mathtt{1}$,
- $\overline{\varphi}_{\mathtt{x}}$ and $\overline{\psi}_{\mathtt{x}}$ are the conditional distributions on $\mathbb{B}^{n-1}$ with the first bit fixed.[1]

The chain rule tells us:

$$\mathsf{kl}(\varphi, \psi) = \mathsf{kl}\left(\begin{bmatrix} p \\ 1-p \end{bmatrix}, \begin{bmatrix} q \\ 1-q \end{bmatrix}\right) + p\,\mathsf{kl}(\overline{\varphi}_{\mathtt{1}}, \overline{\psi}_{\mathtt{1}}) + (1-p)\,\mathsf{kl}(\overline{\varphi}_{\mathtt{0}}, \overline{\psi}_{\mathtt{0}}). \tag{1}$$

[1] $\overline{\varphi}_{\mathtt{1}} := y \mapsto \varphi(\mathtt{1}y)/p$ and $\overline{\varphi}_{\mathtt{0}} := y \mapsto \varphi(\mathtt{0}y)/1-p$.
If $p = 0$ (resp. $p = 1$) $\overline{\varphi}_{\mathtt{1}}$ (resp. $\overline{\varphi}_{\mathtt{0}}$) is assumed to be an arbitrary distribution on $\mathbb{B}^{n-1}$.

# Recursive Definition of KL Divergence

Given

- Let $\mathbb{B} := \{\mathtt{0}, \mathtt{1}\}$ and $\varphi, \psi$ be distributions on $\mathbb{B}^n$,
- $p = \sum_y \varphi(\mathtt{1}y)$ and $q = \sum_y \psi(\mathtt{1}y)$ are the probabilities that the first bit is $\mathtt{1}$,
- $\overline{\varphi}_{\mathtt{x}}$ and $\overline{\psi}_{\mathtt{x}}$ are the conditional distributions on $\mathbb{B}^{n-1}$ with the first bit fixed.[1]

The chain rule tells us:

$$\mathsf{kl}(\varphi, \psi) = \mathsf{kl}\left(\begin{bmatrix} p \\ 1-p \end{bmatrix}, \begin{bmatrix} q \\ 1-q \end{bmatrix}\right) + p\,\mathsf{kl}(\overline{\varphi}_{\mathtt{1}}, \overline{\psi}_{\mathtt{1}}) + (1-p)\,\mathsf{kl}(\overline{\varphi}_{\mathtt{0}}, \overline{\psi}_{\mathtt{0}}). \tag{1}$$

This defines $\mathsf{kl} : \mathcal{D}(\mathbb{B}^n) \times \mathcal{D}(\mathbb{B}^n) \to [0, \infty]$, where $\mathcal{D}$ is the distribution monad.

[1] $\overline{\varphi}_{\mathtt{1}} := y \mapsto \varphi(\mathtt{1}y)/p$ and $\overline{\varphi}_{\mathtt{0}} := y \mapsto \varphi(\mathtt{0}y)/1-p$.
If $p = 0$ (resp. $p = 1$) $\overline{\varphi}_{\mathtt{1}}$ (resp. $\overline{\varphi}_{\mathtt{0}}$) is assumed to be an arbitrary distribution on $\mathbb{B}^{n-1}$.

# Outline


Introduction

String Diagrams

Kullback–Leibler Divergence

Quantitative Diagrammatic Axiomatisation

# KL as an Enrichment

Recall that probabilistic boolean circuits axiomatises $\mathsf{BStoch}^{\otimes}$:

- Objects are natural numbers
- Morphisms $n \to m$ are function $\mathbb{B}^n \to \mathcal{D}(\mathbb{B}^m)$

# KL as an Enrichment

Recall that probabilistic boolean circuits axiomatises $\mathsf{BStoch}^{\otimes}$:

- Objects are natural numbers
- Morphisms $n \to m$ are function $\mathbb{B}^n \to \mathcal{D}(\mathbb{B}^m)$

KL divergence is lifted to homsets $\mathsf{BStoch}^{\otimes}(n, m)$:

$$\overline{\mathsf{kl}}(f, g) = \max_{x \in \mathbb{B}^n} \mathsf{kl}(f(x), g(x)).$$

# KL as an Enrichment

Recall that probabilistic boolean circuits axiomatises $\mathsf{BStoch}^{\otimes}$:

- Objects are natural numbers
- Morphisms $n \to m$ are function $\mathbb{B}^n \to \mathcal{D}(\mathbb{B}^m)$

KL divergence is lifted to homsets $\mathsf{BStoch}^{\otimes}(n, m)$:

$$\overline{\mathsf{kl}}(f, g) = \max_{x \in \mathbb{B}^n} \mathsf{kl}(f(x), g(x)).$$

This yields a $[0, \infty]_+\mathbf{Rel}$-enriched category $\mathsf{BStoch}^{\otimes}_{\mathsf{kl}}$ because

$$\overline{\mathsf{kl}}(f \,;\, g, f' \,;\, g') \leq \overline{\mathsf{kl}}(f, f') + \overline{\mathsf{kl}}(g, g') \qquad \overline{\mathsf{kl}}(f \otimes g, f' \otimes g') \leq \overline{\mathsf{kl}}(f, f') + \overline{\mathsf{kl}}(g, g')$$

**Goal:** Find an axiomatisation of $\mathsf{BStoch}^{\otimes}_{\mathsf{kl}}$.

# Quantitative Diagrammatic Implications

We use quantitative equations from [MPP16] between string diagrams:

$$u \,-\!\boxed{f}\!-\, v =_\varepsilon u \,-\!\boxed{g}\!-\, v \Leftrightarrow d(f,g) \leq \varepsilon$$

[MPP16] Radu Mardare, Prakash Panangaden, and Gordon D. Plotkin. "Quantitative Algebraic Reasoning". In: *Proceedings of the 31st Annual ACM/IEEE Symposium on Logic in Computer Science, LICS '16, New York, NY, USA, July 5-8, 2016*. Ed. by Martin Grohe, Eric Koskinen, and Natarajan Shankar. ACM, 2016, 700–709. DOI: 10.1145/2933575.2934518. URL: https://doi.org/10.1145/2933575.2934518

# Quantitative Diagrammatic Implications

We use quantitative equations from [MPP16] between string diagrams:

$$u \;-\!\boxed{f}\!-\; v =_\varepsilon u \;-\!\boxed{g}\!-\; v \Leftrightarrow d(f,g) \leq \varepsilon$$

We generalize that to implications:

$$\frac{-\!\boxed{f_1}\!- =_{\varepsilon_1} -\!\boxed{g_1}\!- \qquad \ldots \qquad -\!\boxed{f_n}\!- =_{\varepsilon_n} -\!\boxed{g_n}\!-}{-\!\boxed{f}\!- =_\varepsilon -\!\boxed{g}\!-}$$

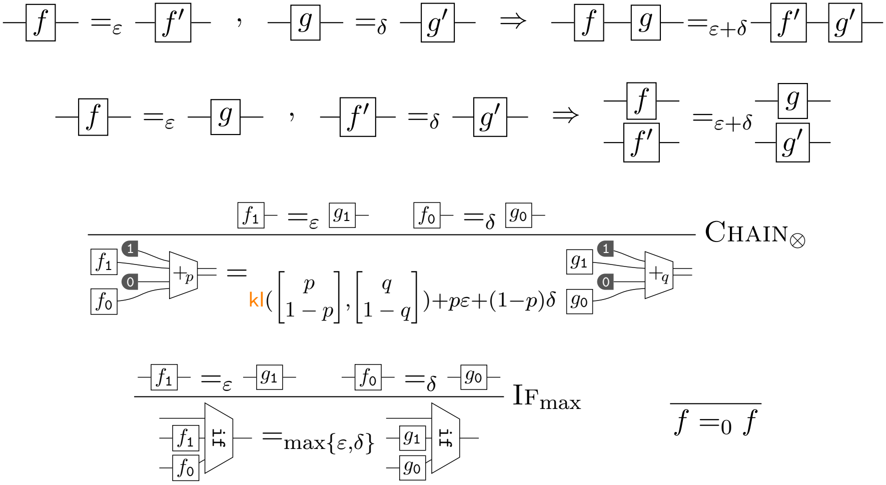
## Theorem

*Given a set of generators and a set of quantitative implications, we can generate the free enriched symmetric monoidal category.*

[MPP16] Radu Mardare, Prakash Panangaden, and Gordon D. Plotkin. "Quantitative Algebraic Reasoning". In: *Proceedings of the 31st Annual ACM/IEEE Symposium on Logic in Computer Science, LICS '16, New York, NY, USA, July 5-8, 2016*. Ed. by Martin Grohe, Eric Koskinen, and Natarajan Shankar. ACM, 2016, 700–709. DOI: 10.1145/2933575.2934518. URL: https://doi.org/10.1145/2933575.2934518

# Axiomatising KL Divergence

With the generators and equations of probabilistic boolean circuits, we add:

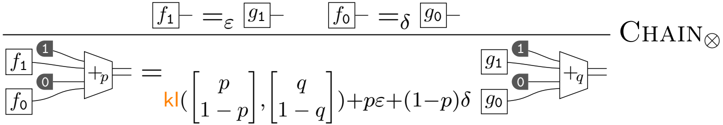


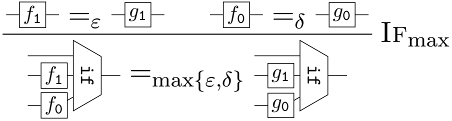
# Axiomatising KL Divergence

With the generators and equations of probabilistic boolean circuits, we add:

$$\boxed{f} =_\varepsilon \boxed{f'} \;,\; \boxed{g} =_\delta \boxed{g'} \;\Rightarrow\; \boxed{f}\!-\!\boxed{g} =_{\varepsilon+\delta} \boxed{f'}\!-\!\boxed{g'}$$

$$\boxed{f} =_\varepsilon \boxed{g} \;,\; \boxed{f'} =_\delta \boxed{g'} \;\Rightarrow\; \begin{matrix}\boxed{f}\\ \boxed{f'}\end{matrix} =_{\varepsilon+\delta} \begin{matrix}\boxed{g}\\ \boxed{g'}\end{matrix}$$




$$\frac{}{f =_0 f}$$

### Theorem

*This theory axiomatises* $\mathsf{BStoch}_{\mathsf{kl}}^{\otimes}$.

# Future Work

## More Examples

- *Already done*: KL divergence, Rényi divergence, total variation
- *Looking into*: Jensen–Shannon divergence, quantum processes
- *Dream*: distance for evaluating probabilistic learning models

## Quantitative Program Logics

- Diagrammatic Hoare logics with tape diagrams in [BDGDL25]
- Quantitative Hoare logics in many papers.
- *Dream*: Quantitative diagrammatic Hoare logics

## Extend the Framework

- *Fibered/Doubled*: to account for distances between in/outputs (Kantorovich)
- *Graded*: for distances that depend on auxiliary information.

[BDGDL25] Filippo Bonchi, Alessandro Di Giorgio, and Elena Di Lavore. "A diagrammatic algebra for program logics". In: *Foundations of software science and computation structures*. Vol. 15691. Lecture Notes in Comput. Sci. Springer, Cham, 2025, pp. 308–330. ISBN: 978-3-031-90896-5; 978-3-031-90897-2. DOI: 10.1007/978-3-031-90897-2_15. URL: https://doi.org/10.1007/978-3-031-90897-2_15

# Differential Privacy Divergence

**Definition**

For distributions $\varphi, \psi \in \mathcal{D}(X)$ and a parameter $\varepsilon \geq 0$,

$$\mathsf{DP}^{\varepsilon}(\varphi, \psi) = \max_{S \subseteq X} (\varphi(S) - e^{\varepsilon} \psi(S)).$$

$f : X \to \mathcal{D}Y$ is $(\varepsilon, \delta)$-**differentially private** if $\sup_{(u,u') \in \mathrm{adj}} \mathsf{DP}^{\varepsilon}(f(u), f(u')) \leq \delta$

We instead consider an enrichment on $\mathsf{BStoch}^{\otimes}$:

$$\mathsf{DP}^{\varepsilon}(f, g) = \max_{x \in \mathbb{B}^n} \mathsf{DP}^{\varepsilon}(f(x), g(x))$$

# Axiomatising DP Divergence

**Facts**

$$\begin{aligned}
\text{(seq. comp.)} \quad & \mathsf{DP}^{\varepsilon+\varepsilon'}(f\,;g, f'\,;g') \le \mathsf{DP}^{\varepsilon}(f, f') + \mathsf{DP}^{\varepsilon'}(g, g') \\
\text{(par. comp.)} \quad & \mathsf{DP}^{\varepsilon+\varepsilon'}(f \otimes g, f' \otimes g') \le \mathsf{DP}^{\varepsilon}(f, f') + \mathsf{DP}^{\varepsilon'}(g, g') \\
\text{(chain rule)} \quad & \mathsf{DP}^{\varepsilon}(\varphi, \psi) = p\mathsf{DP}^{\varepsilon+\log\frac{q}{p}}(\overline{\varphi}_1, \overline{\psi}_1) + (1-p)\mathsf{DP}^{\varepsilon+\log\frac{1-q}{1-p}}(\overline{\varphi}_0, \overline{\psi}_0) \\
\text{(if rule)} \quad & \mathsf{DP}^{\varepsilon}\big(\mathrm{if}(f_1, f_0), \mathrm{if}(g_1, g_0)\big) = \max\left\{\mathsf{DP}^{\varepsilon}(f_1, g_1), \mathsf{DP}^{\varepsilon}(f_0, g_0)\right\}
\end{aligned}$$

We need to change quantitative equations:

$$f =_{\varepsilon,\delta} g \iff \mathsf{DP}^{\varepsilon}(f, g) \le \delta$$

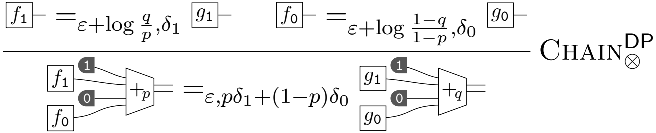
We use distances valued in the quantale $[0, \infty] \times [0, \infty]$.

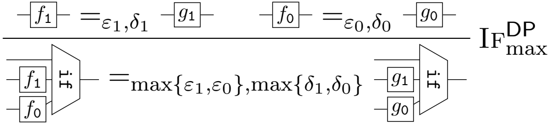
# Axiomatising DP Divergence

With the generators and equations of probabilistic boolean circuits, we add:

$$\boxed{f} =_{\varepsilon,\delta} \boxed{f'} \quad , \quad \boxed{g} =_{\varepsilon',\delta'} \boxed{g'} \quad \Rightarrow \quad \boxed{f}\!-\!\boxed{g} =_{\varepsilon+\varepsilon',\delta+\delta'} \boxed{f'}\!-\!\boxed{g'}$$

$$\boxed{f} =_{\varepsilon,\delta} \boxed{f'} \quad , \quad \boxed{g} =_{\varepsilon',\delta'} \boxed{g'} \quad \Rightarrow \quad \begin{matrix}\boxed{f}\\ \boxed{f'}\end{matrix} =_{\varepsilon+\varepsilon',\delta+\delta'} \begin{matrix}\boxed{g}\\ \boxed{g'}\end{matrix}$$

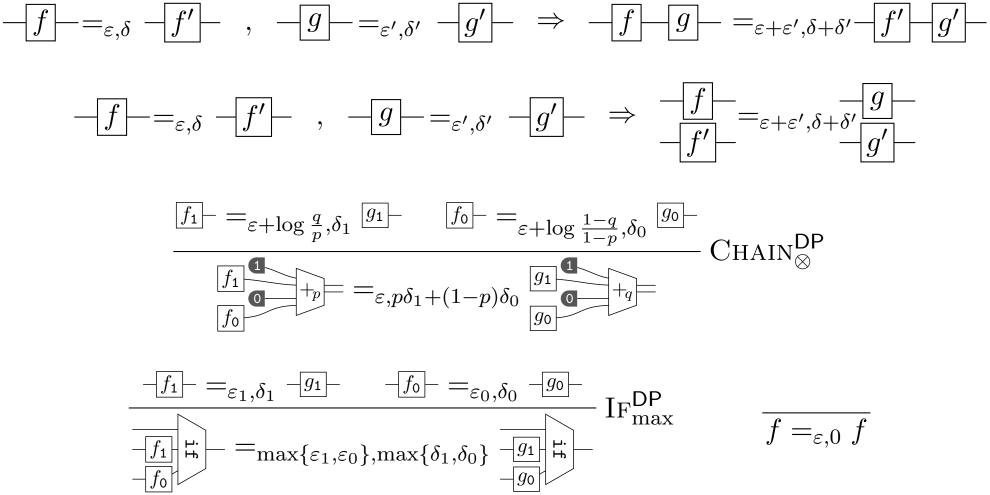



$$\frac{}{f =_{\varepsilon,0} f}$$

# Axiomatising DP Divergence

With the generators and equations of probabilistic boolean circuits, we add:




## Conjecture

This theory axiomatises the enrichment of $\mathsf{BStoch}^{\otimes}$ with $\mathsf{DP}^{\varepsilon}$.

## Questions

- How to handle the real DP with "adjacency" relation on input data?
- Towards a real program logic: tape diagrams [BDGDL25] and imperative categories [Bon+25].

[BDGDL25] Filippo Bonchi, Alessandro Di Giorgio, and Elena Di Lavore. "A diagrammatic algebra for program logics". In: *Foundations of software science and computation structures*. Vol. 15691. Lecture Notes in Comput. Sci. Springer, Cham, 2025, pp. 308–330. ISBN: 978-3-031-90896-5; 978-3-031-90897-2. DOI: 10.1007/978-3-031-90897-2_15. URL: https://doi.org/10.1007/978-3-031-90897-2_15

[Bon+25] Filippo Bonchi et al. *Program Logics via Distributive Monoidal Categories*. 2025. arXiv: 2507.18238 [cs.LO]. URL: https://arxiv.org/abs/2507.18238

# References I


[BDGDL25] Filippo Bonchi, Alessandro Di Giorgio, and Elena Di Lavore. "A diagrammatic algebra for program logics". In: *Foundations of software science and computation structures.* Vol. 15691. Lecture Notes in Comput. Sci. Springer, Cham, 2025, pp. 308–330. ISBN: 978-3-031-90896-5; 978-3-031-90897-2. DOI: 10.1007/978-3-031-90897-2_15. URL: https://doi.org/10.1007/978-3-031-90897-2_15.

[Bon+25] Filippo Bonchi et al. *Program Logics via Distributive Monoidal Categories.* 2025. arXiv: 2507.18238 [cs.LO]. URL: https://arxiv.org/abs/2507.18238.

[Gog+77] J. A. Goguen et al. "Initial Algebra Semantics and Continuous Algebras". In: *J. ACM* 24.1 (1977), 68–95. ISSN: 0004-5411. DOI: 10.1145/321992.321997. URL: https://doi.org/10.1145/321992.321997.

[JS91] André Joyal and Ross Street. "The geometry of tensor calculus. I". In: *Adv. Math.* 88.1 (1991), pp. 55–112. ISSN: 0001-8708,1090-2082. DOI: 10.1016/0001-8708(91)90003-P. URL: https://doi.org/10.1016/0001-8708(91)90003-P.

[MPP16] Radu Mardare, Prakash Panangaden, and Gordon D. Plotkin. "Quantitative Algebraic Reasoning". In: *Proceedings of the 31st Annual ACM/IEEE Symposium on Logic in Computer Science, LICS '16, New York, NY, USA, July 5-8, 2016.* Ed. by Martin Grohe, Eric Koskinen, and Natarajan Shankar. ACM, 2016, 700–709. DOI: 10.1145/2933575.2934518. URL: https://doi.org/10.1145/2933575.2934518.

[Neu51] John von Neumann. “Various Techniques Used in Connection with Random Digits”. In: *National Bureau of Standards Applied Math Series*. Ed. by G. E. Forsythe. Vol. 12. Reprinted in von Neumann’s Collected Works, Volume 5, Pergamon Press, 1963, pp. 768–770. National Bureau of Standards, 1951, pp. 36–38.

[SS71] Dana Scott and Christopher Strachey. “Toward a Mathematical Semantics for Computer Languages”. In: *Proceedings of the Symposium on Computers and Automata*. Vol. 21. 1971.